Relatório 1º projeto ASA 2025/2026

Grupo: AL043 | **Aluno(s):** Carolina Matias (114295); Gonçalo Carrasquinho (114753)

Descrição do Problema e da Solução

O problema apresenta uma cadeia de aminoácidos em que cada um possui um potencial associado. Para cada subsequência da cadeia, o programa deve determinar qual aminoácido remover de forma a maximizar a energia libertada, tendo em conta tanto os potenciais como as afinidades entre as classes dos aminoácidos adjacentes.

A solução construída recorre a um algoritmo de Programação Dinâmica (PD). A PD caracteriza a estrutura ótima analisando, para cada subsequência $[i, j]$, qual o último aminoácido a ser removido, definindo recursivamente a energia máxima possível. A tabela é preenchida de forma bottom-up, garantindo que todas as sub soluções estão calculadas antes de serem usadas. No final, a sequência ótima de remoção é reconstruída por backtracking, garantindo que, em caso de empate de energia, é escolhida a ordem lexicograficamente menor.

Análise Teórica

$$\text{dp[i][j]} = \begin{cases} 0, & se\ i+1 \geq j \\ max(dp[i,k-1] + dp[k+1,l] + contrib(i,k,l)), & caso\ contrário \end{cases}$$

$$contrib(i,k,j) = P[i] * A[i][k] * P[k] + P[k] * A[k][j] * P[j]$$

Leitura dos dados de entrada: Ler n (número de aminoácidos, a sequência de n potenciais (P_i) e as n classes bioquímicas ($C(i)$) —> $O(n)$;

Inicialização das matrizes: Inicializar as matrizes dp[i][j] ($E\,max$) e choice[i][j] (Sequência ótima) para todas as subsequências de comprimento len=1,…,n —> $O(n^2)$;

Aplicação da função solve —> $O(n^3)$:

```
read n; read P[1..n]; read C[1..n]
load affinity table A[5][5]
create DP[1..n][1..n] = 0
create CHOICE[1..n][1..n] = -1

// Dynamic Programming
for len = 1..n:
    for i = 1..n-len+1:
        j := i + len - 1
        leftPot  := (i == 1) ? 1 : P[i-1]
        rightPot := (j == n) ? 1 : P[j+1]
        leftIdx  := (i == 1) ? idx('T') : idx(C[i-1])
        rightIdx := (j == n) ? idx('T') : idx(C[j+1])
        for k = i..j:
            left  := (k == i) ? 0 : DP[i][k-1]
            right := (k == j) ? 0 : DP[k+1][j]
            kIdx := idx(C[k])
            kPot := P[k]
            contrib := leftPot * A[leftIdx][kIdx] * kPot
                     + kPot * A[kIdx][rightIdx] * rightPot
            total := left + contrib + right
            if total ≥ DP[i][j]:
                DP[i][j] := total
                CHOICE[i][j] := k

// Reconstruction
function RECONSTRUCT(i, j):
    if i > j: return []
    k := CHOICE[i][j]
    L := RECONSTRUCT(i, k-1)
    R := RECONSTRUCT(k+1, j)
    return L + R + [k]
sequence = RECONSTRUCT(1, n)
```

- Iterar sobre todos os comprimentos, depois sobre todos os inícios, e por fim sobre todas as escolhas de k dentro de cada subsequência. Calcular $E\,max$ e se for superior ou igual ao valor atual de choice[i][j], atualizamos com o novo total, e k é armazenado em choice[i][j] para reconstrução da sequência.

Grupo: AL043 | **Aluno(s):** Carolina Matias (114295); Gonçalo Carrasquinho (114753)

Reconstrução da sequência ótima: Reconstruir a sequência de remoção da matriz de decisões choice[i][j] usando uma função recursiva —> $O(n)$.

Complexidade global da solução: $O(n^3)$

Avaliação Experimental dos Resultados

Para a avaliação experimental foram geradas 15 instâncias de teste com tamanho incremental de n (número de aminoácidos). O objetivo foi validar o crescimento cúbico da solução, sendo o tempo de execução $O(n^3)$ medido para cada instância gerada.

O gráfico apresenta a relação entre o tempo de execução em função da complexidade teórica ($f(n) = n^3$). Verifica-se uma tendência linear, o que confirma que o tempo de execução do algoritmo é diretamente proporcional à função n^3. Estes resultados demonstram que o algoritmo implementado alinha-se com a análise teórica prevista dada a sua complexidade de $O(n^3)$.

n	tempo (s)
20	0.0208
120	0.0229
220	0.0308
320	0.0546
420	0.0922
520	0.1228
620	0.1874
720	0.2563
820	0.3221
920	0.5213
1020	0.6851
1120	0.8559
1220	1.0645
1320	1.4051
1420	1.7010

